

EXAMEN DE BASE DE DATOS

Diseño de Sistema - Marketplace de Productos Artesanales

Estudiante: 233310

Materia: Base de Datos

Caso: 2-A - Marketplace de Productos Artesanales

Fecha: 5 de febrero de 2026

Versión: Simplificada (5 tablas - Individual)

PARTE 1: PREGUNTAS OBLIGATORIAS

¿Por qué escogí este caso?

Elegí el Caso 2-A del Marketplace de Productos Artesanales por tres razones principales:

- 1. Impacto social:** Me interesa diseñar sistemas que beneficien a comunidades artesanales. Este caso permite crear una plataforma que ayude a artesanos chiapanecos a vender sus productos de manera justa.
- 2. Complejidad técnica apropiada:** El caso presenta desafíos reales e interesantes:
 - Múltiples vendedores en un mismo pedido
 - Preservación de precios históricos
 - Seguimiento independiente de envíos
 - Sistema de reseñas con validaciones
- 3. Aplicabilidad práctica:** Es un modelo de negocio real (similar a Etsy o MercadoLibre) que me permite aprender patrones de diseño aplicables en el mundo profesional del e-commerce.

¿Por qué lo diseñé así?

Decisión 1: 5 Tablas Esenciales

Seleccioné las entidades mínimas necesarias para cumplir todos los requisitos:

artesanos: Gestión de vendedores

productos: Catálogo con estado activo/inactivo

pedidos: Órdenes de compra

detalle_pedido: Tabla pivote que relaciona pedidos con productos

resenas: Sistema de calificaciones

Justificación: Esta estructura cumple todos los requisitos funcionales manteniendo simplicidad y permitiendo futuras extensiones.

Decisión 2: Precio Histórico en detalle_pedido

Guardé precio_unitario en detalle_pedido en lugar de solo referenciar el precio actual.

Justificación: Los pedidos históricos no deben cambiar si el artesano modifica el precio. Esto garantiza: Auditoría correcta, cálculos de comisiones exactos, e integridad de datos transaccionales.

Decisión 3: Estado de Envío Independiente

Incluí estado_envio directamente en detalle_pedido.

Justificación: Cada producto puede tener estado de envío independiente (Don Manuel envía antes que Doña Rosa). Simplifica el modelo sin perder funcionalidad.

Decisión 4: Categorías como CHECK Constraint

Usé un CHECK constraint en lugar de tabla separada:

```
categoria VARCHAR(50) CHECK (categoria IN ('cerámica',
'textil', 'joyería', 'madera', 'otro'))
```

Justificación: Cumple requisito de 5 tablas, validación automática, mejor performance (no requiere JOINS). Sacrificio: menos flexibilidad para agregar categorías (requiere ALTER TABLE).

Decisión 5: Restricción de Reseña Única

Implementé UNIQUE(comprador_email, producto_id).

Justificación: Previene que un usuario deje múltiples reseñas del mismo producto (caso de María que no puede reseñar el collar dos veces).

PARTE 3: SCRIPT SQL - CREACIÓN DE TABLAS

TABLA 1: artesanos

```
CREATE TABLE artesanos (      id SERIAL PRIMARY KEY,      nombre VARCHAR(100)
NOT NULL,      comunidad VARCHAR(100) NOT NULL,      email VARCHAR(100) UNIQUE
NOT NULL,      fecha_registro TIMESTAMP DEFAULT CURRENT_TIMESTAMP );
```

TABLA 2: productos

```
CREATE TABLE productos (      id SERIAL PRIMARY KEY,      artesano_id INTEGER
NOT NULL REFERENCES artesanos(id),      nombre VARCHAR(200) NOT NULL,
descripcion TEXT,      precio_actual DECIMAL(10,2) NOT NULL CHECK
(precio_actual >= 0),      categoria VARCHAR(50) NOT NULL CHECK (categoria IN
('cerámica', 'textil', 'joyería', 'madera', 'otro')),      activo BOOLEAN
DEFAULT true,      fecha_creacion TIMESTAMP DEFAULT CURRENT_TIMESTAMP );
```

TABLA 3: pedidos

```
CREATE TABLE pedidos (      id SERIAL PRIMARY KEY,      comprador_email
VARCHAR(100) NOT NULL,      comprador_nombre VARCHAR(100) NOT NULL,
fecha_pedido TIMESTAMP DEFAULT CURRENT_TIMESTAMP,      fecha_pago TIMESTAMP,
estado_pedido VARCHAR(20) NOT NULL CHECK (estado_pedido IN
```

```
( 'pendiente_pago', 'pagado', 'completado', 'cancelado' )),      total_pedido
DECIMAL(10,2) );
```

TABLA 4: detalle_pedido

```
CREATE TABLE detalle_pedido (      id SERIAL PRIMARY KEY,      pedido_id
INTEGER NOT NULL REFERENCES pedidos(id),      producto_id INTEGER NOT NULL
REFERENCES productos(id),      artesano_id INTEGER NOT NULL REFERENCES
artesanos(id),      precio_unitario DECIMAL(10,2) NOT NULL,  -- PRECIO
HISTÓRICO      cantidad INTEGER NOT NULL DEFAULT 1 CHECK (cantidad > 0),
subtotal DECIMAL(10,2) NOT NULL,      estado_envio VARCHAR(20) DEFAULT
'pendiente' CHECK (estado_envio IN      ('pendiente', 'empacando',
'enviado', 'entregado')),      guia_rastreo VARCHAR(100) );
```

TABLA 5: resenas

```
CREATE TABLE resenas (      id SERIAL PRIMARY KEY,      producto_id INTEGER NOT
NULL REFERENCES productos(id),      comprador_email VARCHAR(100) NOT NULL,
calificacion INTEGER NOT NULL CHECK (calificacion BETWEEN 1 AND 5),
comentario TEXT,      fecha TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
UNIQUE(comprador_email, producto_id)  -- Previene reseñas duplicadas );
```

ÍNDICES PARA OPTIMIZACIÓN

```
CREATE INDEX idx_productos_artesano ON productos(artesano_id); CREATE INDEX
idx_productos_categoria ON productos(categoria); CREATE INDEX
idx_productos_activo ON productos(activo); CREATE INDEX idx_detalle_pedido ON
detalle_pedido(pedido_id); CREATE INDEX idx_detalle_producto ON
detalle_pedido(producto_id); CREATE INDEX idx_detalle_artesano ON
detalle_pedido(artesano_id); CREATE INDEX idx_resenas_producto ON
resenas(producto_id);
```

PARTE 4: CONSULTAS SQL AVANZADAS

Consulta 1: Productos más vendidos por categoría

```
SELECT      p.categoria,      p.nombre,      p.precio_actual,      COUNT(dp.id)
as total_ventas,      SUM(dp.cantidad) as unidades_vendidas,
SUM(dp.subtotal) as ingresos_totales FROM productos p JOIN detalle_pedido dp
ON p.id = dp.producto_id JOIN pedidos ped ON dp.pedido_id = ped.id WHERE
ped.estado_pedido = 'completado' GROUP BY p.id, p.categoria, p.nombre,
p.precio_actual ORDER BY p.categoria, total_ventas DESC;
```

Consulta 2: Reporte de ventas por artesano

```
SELECT      a.nombre as artesano,      a.comunidad,      COUNT(DISTINCT
dp.pedido_id) as pedidos_totales,      SUM(dp.subtotal) as ingresos_brutos,
SUM(dp.subtotal) * 0.88 as ingresos_netos,      SUM(dp.subtotal) * 0.12 as
comision_plataforma FROM artesanos a JOIN detalle_pedido dp ON a.id =
dp.artesano_id JOIN pedidos p ON dp.pedido_id = p.id WHERE p.estado_pedido IN
('pagado', 'completado') GROUP BY a.id, a.nombre, a.comunidad ORDER BY
ingresos_brutos DESC;
```

Consulta 3: Productos con mejor calificación

```
SELECT      p.nombre,      a.nombre as artesano,      AVG(r.calificacion) as
calificacion_promedio,      COUNT(r.id) as total_resenas,      p.precio_actual
FROM productos p JOIN artesanos a ON p.artesano_id = a.id LEFT JOIN resenas r
ON p.id = r.producto_id WHERE p.activo = true GROUP BY p.id, p.nombre,
a.nombre, p.precio_actual HAVING COUNT(r.id) >= 3 ORDER BY
calificacion_promedio DESC, total_resenas DESC;
```

Consulta 4: Estado de pedidos y envíos

```
SELECT      ped.id as pedido_id,      ped.comprador_nombre,
ped.fecha_pedido,      ped.estado_pedido,      p.nombre as producto,
a.nombre as artesano,      dp.estado_envio,      dp.guia_rastreo FROM pedidos
ped JOIN detalle_pedido dp ON ped.id = dp.pedido_id JOIN productos p ON
dp.producto_id = p.id JOIN artesanos a ON dp.artesano_id = a.id WHERE
ped.estado_pedido != 'cancelado' ORDER BY ped.fecha_pedido DESC, ped.id,
dp.id;
```

Consulta 5: Productos activos con artesano y comunidad

```
SELECT      p.nombre as producto,      p.precio_actual,      p.categoria,
a.nombre as artesano,      a.comunidad FROM productos p JOIN artesanos a ON
p.artesano_id = a.id WHERE p.activo = true ORDER BY a.comunidad, p.categoria;
```

Consulta 6: Ingreso total por categoría en periodo

```
SELECT      p.categoria,      SUM(dp.subtotal) as ingreso_total,
COUNT(DISTINCT dp.pedido_id) as pedidos,      SUM(dp.cantidad) as
unidades_vendidas FROM productos p JOIN detalle_pedido dp ON p.id =
dp.producto_id JOIN pedidos ped ON dp.pedido_id = ped.id WHERE
ped.fecha_pedido BETWEEN '2025-01-01' AND '2025-12-31' AND
ped.estado_pedido = 'completado' GROUP BY p.categoria ORDER BY ingreso_total
DESC;
```

Consulta 7: Pedidos con envíos pendientes

```
SELECT DISTINCT      ped.id,      ped.comprador_nombre,      ped.fecha_pedido,
COUNT(CASE WHEN dp.estado_envio != 'entregado' THEN 1 END) as
envios_pendientes FROM pedidos ped JOIN detalle_pedido dp ON ped.id =
dp.pedido_id WHERE ped.estado_pedido != 'cancelado' GROUP BY ped.id,
ped.comprador_nombre, ped.fecha_pedido HAVING COUNT(CASE WHEN dp.estado_envio
!= 'entregado' THEN 1 END) > 0;
```

Consulta 8: Compradores que más han gastado

```
SELECT      comprador_email,      comprador_nombre,      COUNT(DISTINCT id) as
total_pedidos,      SUM(total_pedido) as gasto_total FROM pedidos WHERE
estado_pedido IN ('pagado', 'completado') GROUP BY comprador_email,
comprador_nombre ORDER BY gasto_total DESC LIMIT 10;
```

Consulta 9: Proporción activos/inactivos por artesano

```
SELECT      a.nombre,      COUNT(CASE WHEN p.activo = true THEN 1 END) as
productos_activos,      COUNT(CASE WHEN p.activo = false THEN 1 END) as
productos_inactivos,      ROUND(      COUNT(CASE WHEN p.activo = true THEN
1 END)::DECIMAL /      NULLIF(COUNT(*), 0) * 100,      2      ) as
porcentaje_activos FROM artesanos a LEFT JOIN productos p ON a.id =
p.artesano_id GROUP BY a.id, a.nombre ORDER BY porcentaje_activos DESC;
```

PARTE 5: REPORTES COMO VIEWS

Los reportes implementados como VIEWS permiten consultas reutilizables y optimizadas para los requerimientos del negocio.

VIEW 1: Resumen de ventas por artesano

Descripción: Vista que muestra cuánto ha vendido cada artesano, la comisión retenida (12%) y el pago neto. Solo incluye artesanos con ventas confirmadas.

```
CREATE VIEW vista_ventas_artesanos AS SELECT      a.nombre as artesano,
a.comunidad,      COUNT(DISTINCT dp.pedido_id) as pedidos_totales,
SUM(dp.subtotal) as ingresos_brutos,      SUM(dp.subtotal) * 0.88 as
ingresos_netos,      SUM(dp.subtotal) * 0.12 as comision_plataforma FROM
artesanos a JOIN detalle_pedido dp ON a.id = dp.artesano_id JOIN pedidos p ON
dp.pedido_id = p.id WHERE p.estado_pedido IN ('pagado', 'completado') GROUP
BY a.id, a.nombre, a.comunidad;
```

VIEW 2: Ranking de productos confiables

Descripción: Vista de productos ordenados por calificación promedio. Solo muestra productos con al menos 3 reseñas para garantizar confiabilidad estadística.

Justificación del umbral: Con menos de 3 reseñas, un solo voto extremo (1 o 5 estrellas) distorsiona significativamente el promedio. Tres reseñas es el mínimo para tener una muestra representativa.

```
CREATE VIEW vista_productos_mejor_calificados AS SELECT      p.nombre,
a.nombre as artesano,      AVG(r.calificacion) as calificacion_promedio,
COUNT(r.id) as total_resenas,      p.precio_actual FROM productos p JOIN
artesanos a ON p.artesano_id = a.id LEFT JOIN resenas r ON p.id =
r.producto_id WHERE p.activo = true GROUP BY p.id, p.nombre, a.nombre,
p.precio_actual HAVING COUNT(r.id) >= 3 ORDER BY calificacion_promedio DESC;
```

VIEW 3: Clasificación de velocidad de pago

Descripción: Vista que clasifica pedidos según qué tan rápido pagó el comprador después de crear el pedido.

Justificación de umbrales:

Rápido (≤ 24 horas): Comprador decidido, alta probabilidad de completar transacción

Normal (24-72 horas): Tiempo razonable de reflexión

Tardío (> 72 horas): Posible desinterés o problemas de pago

```
CREATE VIEW vista_pedidos_clasificados AS SELECT      id,
comprador_nombre,      fecha_pedido,      fecha_pago,      total_pedido,
CASE      WHEN fecha_pago - fecha_pedido <= INTERVAL '24 hours' THEN
'Rápido'      WHEN fecha_pago - fecha_pedido <= INTERVAL '72 hours' THEN
'Normal'      ELSE 'Tardío'      END as velocidad_pago FROM pedidos WHERE
estado_pedido != 'cancelado' ORDER BY fecha_pedido DESC;
```

PARTE 6: JUSTIFICACIONES DE DISEÑO

Pregunta 1: Rastreo de envíos independiente

Respuesta: El rastreo se maneja con estado_envio y guia_rastreo directamente en detalle_pedido.

Justificación:

- Cada producto en el pedido puede tener estado de envío independiente
- Cumple el caso: Don Manuel envía rápido (estado: 'entregado') mientras Doña Rosa sigue empacando (estado: 'empacando')
- Simplifica el modelo a 5 tablas (requisito individual)

¿Qué pasa si un pedido tiene productos de 5 artesanos?

- Se crean 5 registros en detalle_pedido, uno por artesano
- Cada registro tiene su propio estado_envio y guia_rastreo
- El pedido NO se marca como 'completado' hasta que TODOS los estados sean 'entregado'

Trade-off aceptado: Sacrifico la capacidad de tener múltiples eventos de rastreo detallados (ej: 'salió de origen', 'en aduana', 'en reparto'). Para el alcance simplificado, un estado único es suficiente.

Pregunta 2: Precio histórico

Respuesta: El precio HISTÓRICO se guarda en detalle_pedido.precio_unitario.

Garantía de no corrupción:

```
-- Al momento de crear el pedido, se CAPTURA el precio vigente: INSERT INTO
detalle_pedido (pedido_id, producto_id, precio_unitario, ...) SELECT 1, 101,
productos.precio_actual, ... FROM productos WHERE id = 101; -- Si después
Don Manuel cambia el precio: UPDATE productos SET precio_actual = 1000 WHERE
id = 101; -- El pedido de María NO cambia: SELECT precio_unitario FROM
detalle_pedido WHERE pedido_id = 1; -- Resultado: 850 (el precio que pagó
María)
```

Dónde queda el dato:

productos.precio_actual: Precio de HOY (mutable)

detalle_pedido.precio_unitario: Precio que pagó el cliente (INMUTABLE)

Esto garantiza:

- Los reportes de ventas sean exactos
- La comisión del 12% se calcule sobre lo realmente cobrado

- Auditoría completa de transacciones

Pregunta 3: Categorías

Decisión: Modelé categorías como CHECK constraint, NO como tabla separada ni texto libre.

```
categoria VARCHAR(50) CHECK (categoria IN ('cerámica',
'textil', 'joyería', 'madera', 'otro'))
```

Justificación:

Cumple requisito de 5 tablas: No agrega tabla adicional

Validación automática: PostgreSQL rechaza valores inválidos

Performance: No requiere JOINS en consultas de productos

Qué sacrifico:

Flexibilidad: Agregar categoría requiere ALTER TABLE (vs simple INSERT en tabla)

Metadatos: No puedo guardar descripción, ícono, o traducciones de categorías

Escalabilidad: Con 50+ categorías, tabla separada sería mejor

Conclusión: Para el alcance del caso (5 categorías estables), CHECK es óptimo. En producción con 50+ categorías dinámicas, usaría tabla separada.

Pregunta 4: Editar reseñas

Cambio requerido: PEQUEÑO

Estado actual (prohíbe duplicados):

```
UNIQUE(comprador_email, producto_id)
```

Opción 1: Permitir UPDATE en vez de INSERT

```
-- Sin cambios en estructura -- Solo cambiar lógica de aplicación: -- Antes
(prohibido): INSERT INTO resenas (producto_id, comprador_email, calificacion,
comentario) VALUES (1, 'maria@ejemplo.com', 5, 'Nuevo comentario'); -- ERROR
por UNIQUE -- Después (permitido): UPDATE resenas SET calificacion = 5,
comentario = 'Comentario editado', fecha = NOW() WHERE producto_id = 1 AND
comprador_email = 'maria@ejemplo.com';
```

Opción 2: Agregar campos de auditoría (opcional):

```
ALTER TABLE resenas ADD COLUMN editado BOOLEAN DEFAULT false; ALTER TABLE
resenas ADD COLUMN fecha_edicion TIMESTAMP;
```

Impacto:

- ✅ Cambio pequeño: No requiere modificar estructura de tablas

Solo lógica: Cambiar INSERT por UPDATE en el código

Opcional: Campos de auditoría si se requiere rastrear ediciones

Conclusión: Es un cambio PEQUEÑO que no afecta el diseño fundamental.

PARTE 7: CASOS DE USO Y PRUEBAS

Caso 1: Pedido de María (del enunciado)

```
-- 1. Crear artesanos INSERT INTO artesanos (nombre, comunidad, email) VALUES
('Manuel García', 'San Cristóbal de las Casas', 'manuel@ejemplo.com'),
('Rosa López', 'Zinacantán', 'rosa@ejemplo.com'); -- 2. Crear productos
INSERT INTO productos (artesano_id, nombre, precio_actual, categoria, activo)
VALUES (1, 'Collar de ámbar', 850.00, 'joyería', true), (1, 'Aretes
de ámbar', 420.00, 'joyería', true), (1, 'Pulsera de ámbar', 310.00,
'joyería', true), (2, 'Huipil tradicional zinacanteco', 1200.00,
'textil', true); -- 3. María crea su pedido INSERT INTO pedidos
(comprador_email, comprador_nombre, estado_pedido, total_pedido) VALUES
('maria@ejemplo.com', 'María Rodríguez', 'pendiente_pago', 2050.00); -- 4.
Agregar productos al pedido (captura precio histórico) INSERT INTO
detalle_pedido (pedido_id, producto_id, artesano_id, precio_unitario,
cantidad, subtotal) VALUES (1, 1, 1, 850.00, 1, 850.00), -- Collar de
Don Manuel (1, 4, 2, 1200.00, 1, 1200.00); -- Huipil de Doña Rosa -- 5.
María paga UPDATE pedidos SET estado_pedido = 'pagado', fecha_pago = NOW()
WHERE id = 1; -- 6. Don Manuel envía rápido UPDATE detalle_pedido SET
estado_envio = 'enviado', guia_rastreo = 'GUIA123' WHERE id = 1; -- 7. Doña
Rosa sigue empacando UPDATE detalle_pedido SET estado_envio = 'empacando'
WHERE id = 2; -- 8. Don Manuel entrega UPDATE detalle_pedido SET
estado_envio = 'entregado' WHERE id = 1; -- 9. María deja reseña del collar
(puede porque ya se entregó) INSERT INTO resenas (producto_id,
comprador_email, calificacion, comentario) VALUES (1, 'maria@ejemplo.com', 5,
'Hermoso trabajo artesanal'); -- 10. María intenta reseñar de nuevo (DEBE
FALLAR) INSERT INTO resenas (producto_id, comprador_email, calificacion,
comentario) VALUES (1, 'maria@ejemplo.com', 5, 'Olvidé poner comentario'); --
ERROR: duplicate key value violates unique constraint -- 11. María NO puede
reseñar el huipil (aún no entregado) -- Lógica de aplicación debe validar:
estado_envio = 'entregado'
```

Caso 2: Pedro busca cerámica inactiva

```
-- 1. Crear Don Luis INSERT INTO artesanos (nombre, comunidad, email) VALUES
('Luis Hernández', 'Amatenango del Valle', 'luis@ejemplo.com'); -- 2.
Producto sin stock (inactivo) INSERT INTO productos (artesano_id, nombre,
precio_actual, categoria, activo) VALUES (3, 'Jarrón de barro negro', 650.00,
'cerámica', false); -- 3. Pedro busca cerámica activa (NO debe aparecer el
jarrón) SELECT * FROM productos WHERE categoria = 'cerámica' AND activo =
true; -- Resultado: 0 filas -- 4. Don Luis reactiva el producto UPDATE
productos SET activo = true WHERE id = 5; -- 5. Ahora sí aparece SELECT *
FROM productos WHERE categoria = 'cerámica' AND activo = true; -- Resultado:
Jarrón de barro negro
```

Caso 3: Lucía cancela pedido pagado


```
-- 1. Pedido de Lucía hace un mes INSERT INTO pedidos (comprador_email,  
comprador_nombre, fecha_pedido, fecha_pago, estado_pedido, total_pedido)  
VALUES ('lucia@ejemplo.com', 'Lucía Fernández', NOW() - INTERVAL '30  
days', NOW() - INTERVAL '30 days', 'pagado', 500.00); -- 2. Lucía  
cancela antes del envío UPDATE pedidos SET estado_pedido = 'cancelado' WHERE  
id = 2; -- 3. El pago NO se elimina (auditoría) SELECT fecha_pago,  
estado_pedido FROM pedidos WHERE id = 2; -- Resultado: fecha_pago sigue  
presente, estado = 'cancelado'
```